

Reviewer Pack — Free Entropy of Read-Twice BP Transition Matrices Bounds Com...

Entry f6e8e84d2355 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 09:44:44 UTC

Free Entropy of Read-Twice BP Transition Matrices Bounds Communication Complexity

Entry ID: f6e8e84d2355 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 09:44:44 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For a read-twice BP P over n variables, let $M(P)$ be its transition matrix with entries normalized to $[0,1]$. Define $\rho(P) = -(1/n) \log \det(\sum_{i=1}^n M(P)^{\otimes i})$. Then $\rho(P) \geq \Omega(\log n)$ for all P , while $\rho(\text{IP_2_trivial_BP}) = O(1)$.

Rationale (proposer's reasoning):

Free entropy quantifies non-commutative randomness; read-twice BPs exhibit structured dependencies that may suppress free entropy, contrasting with the chaotic growth in read-once BPs. The tensor power determinant captures hierarchical state correlations.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 994dc215cc5eed5c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def log_det(matrix):
        n = len(matrix)
        if n == 1:
            return math.log(abs(matrix[0][0]))

        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            sign = (-1) ** j
            det += sign * matrix[0][j] * log_det(submatrix)
        return det

    def tensor_product(matrices):
        if len(matrices) == 1:
            return matrices[0]
        result = matrices[0]
        for i in range(1, len(matrices)):
            new_result = []
            for r1 in result:
                for r2 in matrices[i]:
                    new_row = [sum(a * b for a, b in zip(r1[j], r2[k])) for j in range(len(r1))]
                    new_result.append(new_row)
            result = new_result
        return result

    n = 16
    M_P = [[random.random() for _ in range(n)] for _ in range(n)]
    for row in M_P:
        total = sum(row)
        if total == 0:
            continue
        for i in range(n):
            row[i] /= total
```

```

rho_values = []
for i in range(1, n + 1):
    M_P_i = tensor_product([M_P] * i)
    det = log_det(M_P_i)
    if det <= 0:
        continue
    rho_values.append(-math.log(det) / n)

if not rho_values:
    return {
        "metric_name": "rho",
        "metric_value": 0,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "No valid rho values found"
    }

min_rho = min(rho_values)
metric_value = min_rho
conjecture_holds = min_rho >= 0.3 * math.log(n)
counterexample = "" if conjecture_holds else f"Min rho {min_rho} < 0.3 log n = {0.3 * math.log(n)}"

return {
    "metric_name": "rho",
    "metric_value": metric_value,
    "instances_tested": len(rho_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Min rho < 0.3 log n\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=No valid rho values found")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and optimized parameters for larger instance sizes

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	105963
2	propose	ollama_remote	qwen3:8b	0	0	138460
3	propose	ollama_remote	qwen3:8b	0	0	50888
4	preregistration	ollama_remote	qwen3:8b	0	0	31783
5	novelty	ollama_remote	qwen3:8b	0	0	27562
6	novelty	ollama_remote	qwen3:8b	0	0	21440
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11948
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8384
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9122
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10002
11	judge	ollama_remote	qwen3:8b	0	0	21380

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 436930 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/f6e8e84d2355.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/f6e8e84d2355.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/f6e8e84d2355.tar.gz* (if generated)